VC2

迭代與除錯：讓它變對

VC2 — Iteration & Debugging

八成的卡關，來自「證據給得不夠」。
而你給不出證據，是因為沒有人告訴你去哪裡撈。

Vibe Coding 專班 · 第 2 週 · 混合程度班 | 謝楠楨

這一週結束時，你手上會多出三樣東西

1 一個修好的試算器

- 三個錯全部修好， 21 種組合都算得對
- 而且你說得出「我怎麼知道它修好了」
- 手機尺寸下畫面也不破版
- 這就是 v0.2 ，第 3 週會拿它來寫測試
- 範例 \v0.2 可以對照

2 一份除錯紀錄四欄表

- 現象／證據／我跟 AI 說了什麼／結果
- 證據欄必須是實際貼上的錯誤訊息或輸出
- 不能寫「壞掉了」「怪怪的」
- 三個 bug 各一列
- 這是本週的主要交付物

3 打開開發者工具的習慣

- F12 一按， Console 分頁一看
- 從「完全沒線索」變成「看得到證據」
- 這一個動作會改變你之後所有的除錯經驗
- 第 4 週部署除錯也要用它
- 學會這個，比修好三個 bug 更重要

這一週的主角不是「修好三個 bug 」，而是「你怎麼描述問題」。 AI 修 bug 的能力已經很強了——真正的瓶頸在於，你能不能把「怪怪的」變成一段它能動手的具體描述。

本週地圖：哪幾頁要讀、哪幾頁要動手

頁	段落	怎麼用	預估時間
4–9	PART 01　先規劃，還是先許願	讀。六步工作流與三種提示句型	15 分
10–16	PART 02　讀懂錯誤訊息與開發者工具	讀 + 跟著把 F12 按開來看一次	25 分
17–23	PART 03　找出問題在哪	讀。最小重現例、二分法、什麼時候該重來	18 分
24–45	PART 04　動手做（Step 1–7）	打開練習版本，跟著做。這是本週的主體	90–120 分
46–47	卡住了？照症狀查	不必從頭讀，卡住時再翻	—
48	常見的四個誤解	做完之後回頭看一次	5 分
49–51	自我檢核、本週交付與回顧	交作業前一定要看	10 分

開始動手之前，請先下載 範例 \W2_ 有三個 bug 的版本。整個 PART 04 都在修它。裡面有一份「學生用說明 .md」，答案在「教師用解答 .md」——請先自己找，找不到再看。

PART 01

先規劃，還是先許願

Plan or Wish

「想到什麼就叫 AI 做什麼」不是 Vibe Coding，那是許願。
有沒有先規劃，決定了你是在開發，還是在碰運氣。

六步工作流：從想法到上線

1

① 規劃與規格

先寫清楚

2

② 生成

讓 AI

3

③ 驗證

自己確認

4

④ 迭代

描述現象

5

⑤ 版本控制

存下可以

6

⑥ 部署

讓別人打

這一週在第 ④ 步。第 ① 步你上週做過了（五行規格），第 ③ 步是下一週的整份講義。注意 ③ 與 ④ 是一個循環——驗證發現問題就回去迭代，迭代完再驗證一次，通常要繞三到五圈。

每一步做什麼、產出什麼、最常見的錯

步驟	做什麼	產出	最常見的錯
① **規劃與規格**	寫下做什麼／給誰／輸入／輸出／技術限制	五行規格 ＋ 驗收條件	跳過這一步直接叫它做，它會自己猜功能
② **生成**	把規格貼給 AI，先讀它的實作計畫再讓它動手	第一版可以跑的東西	沒讀它的計畫就讓它寫，理解錯了要整份重來
③ **驗證**	自己點過每一種情況，對照驗收條件	確認清單	只點自己最熟的那一兩種
④ **迭代**	描述現象、給證據，一次修一個	修好的版本 ＋ 除錯紀錄	一次講三個問題；或只說「壞了」不給證據
⑤ **版本控制**	每個可運行的狀態立刻存一個點	commit 紀錄	改壞了才發現沒有可以回去的地方
⑥ **部署**	推上去、拿別人的裝置開一次	公開網址	只在自己電腦上開過就說做完了

> 第 ④ 步的兩個常見錯誤，正好是這一週要治的：「一次講三個問題」會讓你分不清是哪一句話造成哪個改變；「只說壞了」則讓 AI 只能用猜的。

三種必備的提示句型

1 System 定調　說一次就好

- 在專案根目錄放一份規則檔
- Claude Code 用 CLAUDE.md 、Cursor 用 .cursorrules
- 寫進去：技術棧、風格、命名、不可以做什麼
- 效果：不必每次重複交代，產出穩定很多
- 例：「這是單頁靜態網頁，不要引入任何框架」

2 任務　一次一件事

- 壞例子：「幫我做一個病人資料管理系統」
- 好例子：「閱讀 @ 規格 .md 第 3 節，實作篩選功能。

  只改 calculator.js ，不要動 index.html 」
- 三個要素：明確參照、單一目標、明確範圍
- 範圍限制最重要——它防止 AI 順手改別的地方

3 修正　給證據不給形容詞

- 這一週的主角，下一頁詳細講
- 壞例子：「算錯了，修一下」
- 好例子：「我選第 5 級 + 低收，自負額顯示 3,856 。

  低收的部分負擔比率是 0% ，應該顯示 0 」
- 現象 + 證據 + 正確答案，三件都要有

三種句型的共同原則：讓 AI 少猜一點。它的能力已經很強，錯誤多半來自「你沒說清楚，它只好猜」。而猜錯的成本，最後是你在承擔。

實戰 Prompt 範本（四種情境，可直接照抄）

情境	可以直接照抄的範本
新功能開發	閱讀 @ 規格 .md 第 N 節，實作【功能名稱】。 只修改【檔案路徑】，不要更動其他檔案。完成後說明你改了什麼。
除錯（本週主力）	現象：我做了【什麼操作】，看到【什麼結果】。 應該是：【正確答案】，依據是【出處】。 〔如果有錯誤訊息，整段貼在這裡〕 請只修改【檔案】。動手之前先說明你認為問題出在哪一行、為什麼。
理解既有的程式碼	請用完全不懂程式的人也看得懂的話，說明【檔名】的【函式名】在做什麼。 不要改動任何邏輯，只加中文註解。
確認它真的改對了	請說明你實際改了哪幾行，改前與改後分別是什麼。 另外，這個修改會不會影響到【其他情況】？

第二個範本是本週會用最多次的，建議直接存起來。它有四個要素：現象、正確答案與依據、證據、範圍限制——四個都不能少。後面 Step 2-3 會帶你實際填一次。

修正 Prompt：給證據，不要給形容詞

	× 形容詞（AI 只能猜）	○ 證據（AI 可以動手）
低收 自負額錯	「算錯了」 → 「金額 怪怪的」 → AI 只能 到處亂改 → 改完你也 不知道對不對	「第 5 級 ＋低收」 → 「顯示 3,856」 → 低收比率 0% → 應顯示 0 → AI 直接 改對那一行
第 8 級 上限錯	「有些等級 不對」 → 「你再 檢查一下」 → 它會回 「已修正」 → 但你不知道 它改了什麼	選第 8 級 → 「上限顯示 32,090」 → 「基準應為 36,180」 → 「請說明你 改了哪一行」
按了 沒反應	「按鈕壞了」 → 「都不能用」 → 它看不到 你的螢幕 → 來回三次 還在猜	「不選身分別 按計算」 → 「畫面 完全沒變」 → 「Console： 〔貼錯誤 訊息〕」 → 一次就中

一句話記住：AI 看不到你的螢幕，你是它唯一的感官。你貼的每一個字，就是它能掌握的全部資訊。「怪怪的」對它來說，跟什麼都沒說是一樣的。

PART 02

讀懂錯誤訊息

Reading Error Messages

錯誤訊息看起來很嚇人，其實只有四個部位。

而且新手常犯的錯，是從最上面一行開始讀。

一則錯誤訊息該從哪裡開始看

瀏覽器 Console 裡實際看到的樣子

Console

Uncaught TypeError: Cannot read properties of undefined (reading 'rate') ①

at calculate (calculator.js:42:26) ③

at HTMLButtonElement.<anonymous> (index.html:88:17) ④

（再往下是瀏覽器自己的程式，通常不用看）

建議的閱讀順序：② 具體訊息 → ③ 位置 → ① 類型 → ④ 堆疊。
先看②因為它最具體；再看③知道去哪裡看；①和④是補充脈絡。
新手常犯的錯是從最上面一行開始讀，然後被 Uncaught TypeError 這幾個字嚇退。

① 錯誤類型　TypeError

告訴你是「哪一類」問題。 TypeError ＝拿到的不是預期的型別
這一格決定你要問 AI 什麼方向

② 具體訊息　reading 'rate'

最有資訊量的一格：想讀 rate 欄位，但那個東西是 undefined
翻成人話：某個查表沒查到，回來的是空的

③ 出事的位置　calculator.js:42

第一個要打開的地方： calculator.js 第 42 行第 26 個字
貼給 AI 時務必連這一行一起貼

④ 呼叫堆疊　往下是誰叫它的

由上往下＝由內往外：第 42 行是被下一行的按鈕事件叫起來的
用來回答「什麼操作觸發了它」；只看含自己檔名的那幾行

給 AI 的時候整段一起貼，不要只貼第一行。這一整段就是「證據」——你貼得越完整， AI 猜得越少。這就是上一頁那句話的具體做法。

四種最常遇到的錯誤類型，各自代表什麼

錯誤類型	白話的意思	在這個專案裡可能是什麼	怎麼問 AI
TypeError	拿到的東西不是我以為的那種	查表沒查到，回來的是 undefined，卻還想讀它的欄位	「〔貼整段〕這一行的 X 為什麼會是 undefined？什麼情況會發生？」
ReferenceError	你叫的那個名字，我不認識	calculator.js 沒被載入，或函式名拼錯	「〔貼整段〕為什麼會找不到 calculate？」
SyntaxError	這段文字我看不懂，讀不下去	少一個括號、少一個引號。這一類整份檔案都不會執行	「〔貼整段 + 那一行前後五行〕哪裡的語法錯了？」
完全沒有錯誤訊息	程式跑得好好的，只是算錯	低收自負額算成 1,205、第 8 級上限算成 32,090	不能問 AI「哪裡錯了」，要告訴它「應該是多少」

最後一列最重要，而且最常見。沒有錯誤訊息的錯誤，AI 完全無從察覺——它只知道程式跑完了。這種錯只有「知道正確答案應該是多少」的人抓得到，也就是你。

開發者工具怎麼打開

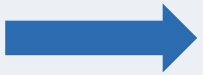
環境	怎麼開	開起來之後先看哪裡
Windows　Chrome / Edge	按 F12。沒反應的話按 Ctrl + Shift + I	上排分頁選「Console」。有紅字就是有錯
macOS　Chrome	按 Option + Command + I（⌥⌘I）	同上
macOS　Safari	先到 Safari → 設定 → 進階 → 勾「顯示開發選單」， 再按 Option + Command + C	同上。Safari 預設是關的，要先開
Firefox（Win / Mac）	F12；或 Ctrl + Shift + K（Mac 是 ⌥⌘K）直接開 Console	同上。分頁名稱叫「主控台」
手機模式（模擬）	開發者工具左上角有一個手機／平板圖示，點它。 Windows 快捷鍵 Ctrl + Shift + M	上方會出現裝置選單，選 iPhone 或自訂寬度 390
A 軌　瀏覽器工具	工具內建的預覽視窗通常有自己的錯誤面板。 但仍建議把預覽開成獨立分頁再按 F12	獨立分頁才有完整的開發者工具

> 第一次按 F12 會被畫面上突然出現的一大塊東西嚇到，這是正常的。你只需要用到上排的三個分頁：Console、Network、Elements。其他的暫時當作不存在。

Console 裡的三種顏色，分別代表什麼

顏色

顏色	意思
紅字	錯誤 Error
黃字	警告 Warning
灰／白字	一般訊息 Log
（空的）	什麼都沒發生



處理方式

你該怎麼辦
一定要看。程式在這裡出事了
多半可以忽略。例：某個寫法已過時
程式自己印出來的訊息，通常無害
不代表沒問題——只代表沒當掉

最重要的一句：沒有紅字 ≠ 算得對

本週三個 bug 裡，只有一個會留下紅字。另外兩個安安靜靜地顯示了錯的金額， Console 一片乾淨

紅字很多看不出哪一則

先按右上角的垃圾桶圖示清空，然後只做「按一次計算」這一個動作，看新出現什麼

黃字要不要理

課堂專案可以先忽略。它多半在說「這個寫法以後會被淘汰」，不影響現在能不能跑

貼給 AI 時要貼多少

整段紅字，含下面 at ... 開頭的那幾行。只貼第一行等於丟掉了「在哪裡出事」這個最有用的資訊

建立一個習慣：每次操作前先清空 Console，操作後看一眼。這兩個動作加起來不到三秒，但它把「完全沒線索」變成「有證據可貼」。

三個分頁，各回答一個問題

1 Console 「有沒有出錯？」

- 九成的時候你只需要這一個
- 紅字＝錯誤，黃字＝警告（多半可忽略）
- 沒有紅字，代表「程式沒當掉」，不代表「算得對」
- 右上角有垃圾桶圖示可以清空
- 清空之後再操作一次，看新出現什麼

2 Network 「資料拿到了嗎？」

- 看網頁向外要了哪些東西、成功了沒
- 這個專案不對外連線，用得少
- 但第 5 週接外部 API 時會天天用
- 看狀態碼：200 是成功、404 是找不到、500 是對方伺服器出錯
- 檔案載入失敗也會出現在這裡

3 Elements 「畫面為什麼長這樣？」

- 看目前畫面實際的結構與套用的樣式
- 版面跑掉、字太小、東西沒對齊時用
- 點左上角的箭頭圖示，再點畫面上的元素
- 右邊會顯示它套了哪些 CSS
- 可以直接改改看，重新整理就還原
- 適合用來確認「是不是這一條規則造成的」

本週會用到 Console（找 Bug 3）與 Elements（修手機破版）。Network 這一週用不到，但先知道它在做什麼——第 5 週會用得很兇。

A 軌補充：瀏覽器工具裡的錯誤要去哪裡看

你用的工具	錯誤訊息會出現在哪	建議的做法
Lovable / Bolt.new 等對話式工具	① 工具自己的對話區會提示「發生錯誤」 ② 預覽視窗旁邊通常有一個錯誤或 Console 面板	把預覽開成獨立分頁（通常有「在新分頁開啟」或「Open in new tab」按鈕），再按 F12。 這樣才有完整的開發者工具，跟 B 軌完全一樣
線上編輯器（Replit 等）	內建的 Console 面板，通常在下方或右側	夠用，但手機模式模擬通常沒有—— 要看手機版仍建議開獨立分頁
兩軌共通	不論用哪種工具，最後都會部署成一個網址（第 4 週）	部署之後用瀏覽器打開那個網址，按 F12—— 這是所有工具都一樣的做法，也是最可靠的

為什麼堅持 A 軌也要學 F12：工具內建的面板各家不同、隨時會改版，而瀏覽器的開發者工具十幾年來都長一樣，而且到哪裡都能用。學一次，用一輩子。

PART 03

找出問題在哪

Locating the Problem

修 bug 的難處從來不是「怎麼修」——那是 AI 的強項。
難的是「問題到底在哪一段」。

三種故障樣態，處理方式完全不同

1 會報錯　最好處理

- Console 出現紅字，程式當場停下來
- 你有現成的證據可以貼給 AI
- 本專案的 Bug 3 屬於這一類
- 處理：整段錯誤訊息貼過去，通常一次就中
- 難處只在於「你要記得去看 Console」

2 沉默失敗　最陰險

- 按了沒反應，或畫面維持舊的數字不變
- 使用者完全不會發現自己看的是過期結果
- Console 可能有紅字，也可能什麼都沒有
- 處理：先打開 Console 再操作一次
- 沒有紅字的話，就要用二分法找

3 數字錯　最危險

- 程式跑得好好的，只是算出來的數字不對
- 不會報錯、不會變色、不會提醒你
- Bug 1（低收）與 Bug 2（第 8 級）都是這一類
- 處理：只能靠「你知道正確答案應該是多少」
- AI 完全無從察覺這種錯

醫療場域最該警惕的是第 3 類。它不會造成任何技術上的異常，卻會用跟正確答案一模一樣的樣子顯示出來——而使用者正是因為不知道答案，才來用這個工具的。

最小重現例：把「整個都壞了」縮小到一句話



最小重現例的標準格式：「做〔這個操作〕，輸入〔這個值〕，就會〔這個結果〕；改成〔那個值〕就正常。」——三個空格填滿，你的問題就已經解決一半了。而且這個過程你自己就做得到，不需要懂程式。

不知道哪裡壞掉時：二分法

畫面出了問題，但不知道是哪一段造成的

1. Console 有沒有紅字?
   - 是 → 有 → 直接看檔名與行號，問題位置已經告訴你了
     整段貼給 AI，這一題就結束了
     這是最幸運的情況，先檢查這個
   - 否 ↓
2. 是「算的」還是「顯示的」出問題?
   - 是 → 判斷方法：把同樣的輸入自己手算一次
     手算跟畫面一樣 → 算是對的，問題在顯示（格式、對齊、單位）
     手算跟畫面不一樣 → 問題在計算，去看 calculator.js
   - 否 ↓
3. 是所有情況都錯，還是只有某些情況?
   - 是 → 全部都錯 → 通常是一個地方錯導致整排錯，找共同的那一段
     只有某些錯 → 找出「會錯」與「不會錯」的分界點
     分界點常常就是邊界值（最小、最大、零、空）
   - 否 ↓
4. 還是找不到 → 砍一半
   - → 請 AI 把功能暫時拆成兩半，先確認前半段的結果對不對
     例：先只顯示「查到的上限」，不算自負額。上限對嗎?
     對 → 問題在後半段；不對 → 問題在前半段。再砍一次

二分法是所有除錯方法裡最可靠的一個，因為它不需要任何猜測——每砍一次，範圍就少一半。十幾行的程式碼砍三、四次就到底了。這個方法你在查任何問題時都用得上，不限於程式。

AI 卡住時的四個手法

1 給足證據

- 貼完整的錯誤訊息，不要只說「壞了」
- 貼實際的輸出與你預期的輸出
- 說明你做了什麼操作才出現這個問題
- AI 看不到你的螢幕——你是它唯一的感官
- 八成的卡關來自證據給得不夠

2 要它先解釋再動手

- 「動手之前，請先說明你認為問題出在哪裡」
- 它的解釋錯了，你當場就攔得下來
- 比讓它改完再發現理解錯了省事十倍
- 解釋看不懂時，追問「舉一個實際例子」
- 這也是第 3 週「讀懂程式碼」的暖身

3 換個問法

- 同一句話重複三次不會有不同結果
- 改成問「有哪幾種可能的原因？」
- 或「如果你要排除這個問題，會先檢查什麼？」
- 或直接開一個新對話重貼一次
- 對話太長時它會忘記前面（M7 的上下文限制）

還有第四個手法：退回上一個可用的版本重來。這是第 4 週的內容——如果你連續改了五次都越改越糟，最有效率的做法通常是回到還能跑的那一版，重新用更好的描述再講一次。

描述現象，不要下指令：三種句型與實際範例

句型	什麼時候用	實際可以照抄的範例
描述現象	發現問題時的第一句話。 最重要的一種	我選第 5 級 + 低收入戶，自負額顯示 3,856 。 依長照給付基準，低收入戶部分負擔為 0% ，應該顯示 0 。
要它先解釋	問題看起來不單純、 或你想順便學會時	動手之前，請先說明你認為問題出在哪一行、為什麼。 我確認之後再請你修改。
限制範圍	怕它順手改壞別的地方時。 本週每一次修改都建議加	請只修改 calculator.js 的 IDENTITY_RATES ， 不要更動其他任何檔案或函式。完成後說明你改了什麼。
確認修好	它說「已修正」之後	請說明你實際改了哪幾行，改前與改後分別是什麼。 另外，這個修改會不會影響到其他等級或身分別?

第四種句型最常被跳過，卻最重要。 AI 說「已修正」不代表真的修對了——要它說出「改了哪幾行、改前改後是什麼」，你才有辦法自己核對。這也是除錯紀錄第四欄的內容來源。

越改越糟的時候：什麼時候該停下來重來

你已經來回請 AI 修了三、四次，但問題還在或變得更糟

1. 這幾次修改，它有沒有動到你沒要求的檔案？
   - 是 → 有 → 立刻停。先要它列出所有改動，還原不必要的部分
     再從還能跑的那一版重新開始，這次在指令裡寫明範圍限制
     這是最常見的失控原因，而且越拖越難收拾
   - 否 ↓
2. 你講得出「現在到底是哪裡不對」嗎？
   - 是 → 講不出來 → 停止修改。回頭做最小重現例（第 19 頁）
     把「哪一個操作、哪一個輸入、會出現什麼」寫清楚再繼續
     你自己都描述不清楚的問題， AI 只能亂猜
   - 否 ↓
3. 原本會動的功能，現在也壞了嗎？
   - 是 → 有 → 果斷退回。改壞的範圍已經超過原本的問題
     本週的土法：把備份資料夾複製回來（所以每次大改前要先備份）
     第 4 週會學 git revert ，那是正規做法
   - 否 ↓
4. 以上都不是，只是還沒改對
   - → 換個問法再試一次：「有哪幾種可能的原因？請各說明怎麼確認」
     或開一個新對話，重貼完整檔案（對話太長它會忘記前面）
     再兩次還是不行，就退回重來——沉沒成本不值得留戀

一個實用的門檻：同一個問題來回超過四次還沒解決，就不要再修了。退回上一個可用的版本，把問題重新描述一次，通常比繼續往下修快得多——而且你會得到一份乾淨的程式碼，而不是被改得面目全非的那一份。

PART 04

動手做： Step 1–7

Hands-on

請先下載 範例 \W2_ 有三個 bug 的版本，整個 PART 04 都在修它。
三個 bug 的難度是拉開的——一個一眼看到、一個要試邊界、一個要開 F12 。

Step 1　先自己點一遍，什麼都不要修

你現在要做什麼　▶　打開 W2 練習版本的 index.html，把每一種組合點過去

你要做的事

1. 雙擊 index.html 打開
2. 選第 5 級 + 一般戶 → 按計算 → 記下四個數字
3. 換成第 5 級 + 低收入戶 → 按計算 → 記下數字
4. 換成第 8 級 + 一般戶 → 按計算 → 記下數字
5. 身分別留在「請選擇」→ 按計算 → 看會怎樣

自己先算一次（不要看程式碼）

第 5 級上限 24,100　一般戶 16% → 自負額 ?
第 5 級上限 24,100　低收 0%　→ 自負額 ?
第 8 級上限 ?
（給付基準見範例的 README，或 VC1 講義）

你應該看到什麼

畫面會正常出現數字，
看起來完全沒問題。
但你手算的結果會跟畫面對不上——
至少有兩組對不上。
第 4 步（不選身分別）則是
按了完全沒反應。

沒看到的話，先檢查這幾件事

① 打不開／畫面空白
→ 確認 index.html 與 calculator.js
在同一個資料夾
② 不知道正確答案是多少
→ 上限 × 比率。一般戶 16%、
中低收 5%、低收 0%

這一步刻意不讓你看程式碼。理由是：真實情況下你也不會先看程式碼——你是先發現「數字不對」，才回頭去找原因。這個順序要練起來。

Step 1-2　把 F12 按開來，先熟悉一下

你現在要做什麼　▶　在剛才那個畫面上按 F12（macOS 是 ⌥⌘I），把 Console 分頁找出來

Windows　Chrome / Edge

F12　或　Ctrl + Shift + I
上排選「Console」分頁

macOS

Chrome： Option + Command + I
Safari： 先到 設定 → 進階 → 勾「顯示開發選單」，
　　再按 Option + Command + C

接著做這件事

把 Console 清空（右上角垃圾桶圖示），
然後身分別留「請選擇」，按一次計算。

你應該看到什麼

Console 出現一段紅字，
開頭是：
　Uncaught TypeError: Cannot read
　properties of undefined (reading 'rate')
下面還有幾行 at ... 開頭的。
這就是 Bug 3 的證據——
但先不要修， Step 4 才處理它。

沒看到的話，先檢查這幾件事

① 按 F12 沒反應
　→ 試 Ctrl + Shift + I；
　　或從瀏覽器選單 → 更多工具
② 找不到 Console 分頁
　→ 分頁列可能被收起來了，
　　點 >> 展開
③ 沒有紅字 → 先按垃圾桶清空再按一次

恭喜，你剛剛完成了本週最重要的一個動作。「畫面沒反應」原本是完全沒有線索的死路，現在你手上有了一段可以貼給 AI 的證據。這個習慣值得帶到往後的每一個專案。

Step 1-3　順手把手機模式也開起來看一次

你現在要做什麼　▶　在開發者工具左上角找到手機／平板圖示，點下去

怎麼切換

開發者工具左上角有一個「手機＋平板」的小圖示。
Windows　快捷鍵：Ctrl + Shift + M

點下去之後，畫面上方會出現裝置選單。
選 iPhone SE，或直接把寬度輸入 390。

你應該看到什麼

畫面會超出螢幕：
卡片被切掉一部分，
或是要左右拖曳才看得到完整內容。
字也會變得很小。
這就是本週的附帶任務（Step 5）。
先看到就好，還不用修。

沒看到的話，先檢查這幾件事

① 找不到那個圖示
　→ 用 Ctrl + Shift + M
② 切了之後畫面看起來正常
　→ 把寬度手動設成 390 再看一次
③ 想關掉 → 再按一次同一個圖示

為什麼要在這裡就看一次：讓你先建立「本機好好的，不代表手機上沒問題」這個直覺。這是測試永遠蓋不到、只有換裝置才看得見的一整類問題。

Step 2　帶著找：第一個 bug（一眼看到的那種）

你現在要做什麼　▶　回到 Step 1 你手算的三組數字，找出對不上的第一組

你手算的答案應該是

```
第 5 級 × 一般戶   24,100 × 16%  = 3,856
第 5 級 × 低收     24,100 × 0%   = 0
第 8 級 × 一般戶   36,180 × 16%  = 5,789
```

畫面上實際顯示的

```
第 5 級 × 一般戶   3,856    ○ 一樣
第 5 級 × 低收     1,205    × 對不上
第 8 級 × 一般戶   5,134    × 對不上（上限也錯）
```

你應該看到什麼

你會發現至少兩組對不上。
先處理低收那一組——
它最單純，而且你很確定答案是 0。
先不要看程式碼，
也先不要問 AI「哪裡錯了」。
下一頁先做一件事：確認你自己是對的。

沒看到的話，先檢查這幾件事

① 我的手算跟畫面一樣，沒對不上
　→ 檢查比率：低收是 0%，不是 5%
② 不確定 24,100 這個數字對不對
　→ 範例的 README 有完整的額度表，
　　也可以查衛福部的給付基準

注意這一步的順序：先自己算，再比對畫面。反過來做（先看畫面再想「應該是多少」）很容易被畫面上的數字帶著走——這是人類認知的通病，跟你懂不懂程式無關。

Step 2-2　先確認「你是對的」，再去說「它是錯的」

你現在要做什麼　▶　在跟 AI 說任何話之前，先把你的依據找出來

要找到的三件事

① 正確答案是多少?　→ 0
② 依據是什麼?　→ 低收入戶部分負擔比率 0%
③ 出處在哪?　→ 衛福部長照給付基準／範例 README

為什麼這一步不能省

如果你搞錯了， AI 會很配合地把「對的」改成「錯的」，
而且完全不會質疑你。

AI 沒有立場，它只會照你說的做。

你應該看到什麼

三件事都寫得出來。
特別是第 ③ 項——
「我查過基準」跟「我覺得應該是」
是完全不同的兩件事。
這一句也會直接變成
除錯紀錄表的「證據」欄。

沒看到的話，先檢查這幾件事

① 找不到依據，只是「感覺不對」
→ 先去查。查不到就先跳過這一個，
處理你確定的那一個
② 查到的說法不一致
→ 以官方公告為準，並把這個
不確定寫進紀錄

這一頁是整份講義的一個小重點： Vibe Coding 讓「改」變得非常快，快到你來不及確認自己是不是要求對了。而 AI 不會攔你。把關的只有你自己。

Step 2-3　怎麼跟 AI 說：現象 + 證據 + 正確答案

你現在要做什麼　▶　把下面這段照抄，貼給 AI

可以直接照抄

我在測試長照給付試算器，發現一個計算錯誤。

現象：我選 CMS 第 5 級 + 低收入戶，按計算後
「民眾自負額」顯示 NT$ 1,205 。

應該是：依長照給付基準，低收入戶的部分負擔比率為 0% ，
所以自負額應該是 0 。

請只修改 calculator.js ，不要更動其他檔案。
動手之前先說明你認為問題出在哪一行、為什麼。

你應該看到什麼

它會先回你一段說明，
指出 IDENTITY_RATES 裡低收入戶的
rate 被寫成 0.05 ，應該是 0 。
先讀這一段再讓它動手——
說明錯了就在這裡攔下來。
確認無誤後再說「請修改」。

沒看到的話，先檢查這幾件事

① 它直接改了沒有先說明
→ 沒關係，但要它補說明改了哪幾行
② 它說要改別的地方
→ 停。先問「為什麼是那裡？」
它可能誤解了你的描述
③ 它順手改了其他檔案
→ 要它還原，重申只改 calculator.js

這段話裡有四個要素：現象（我做了什麼、看到什麼）、應該是（正確答案與依據）、範圍限制（只改哪個檔）、先解釋再動手。四個都不能少——少哪一個，前面對應的那一種災難就會發生。

Step 2-4　怎麼確認它真的修好了

你現在要做什麼　▶　它說「已修正」之後，做這三件事（不要只相信它）

三件事

① 要它說明：「請說明你實際改了哪幾行，
改前與改後分別是什麼？」

② 自己回畫面點一次：第 5 級 + 低收 → 應顯示 0

③ 點一個「不該被影響」的組合：
第 5 級 + 一般戶 → 應該還是 3,856

你應該看到什麼

① 它會說：把 low 的 rate
從 0.05 改成 0，只有這一行。
② 畫面顯示自負額 NT$ 0。
③ 一般戶仍然是 3,856——
沒有被改壞。
三件都對，Bug 1 才算結束。

沒看到的話，先檢查這幾件事

① 一般戶也變成 0 了
→ 典型的「改 A 壞 B」。
告訴它「一般戶應維持 16%」
② 它說改了三、四個地方
→ 要它說明每一處的理由，
可疑的就要它還原

第 ③ 件事就是「回歸測試」的人工版本——確認你修的東西沒有把別的地方弄壞。第 3 週會把這件事自動化成 test.html，到時候三秒就能驗完 29 項。這一週先用手做，你才會知道自動化在替你省什麼。

Step 3　自己找：第二個 bug （要試邊界才會發現）

你現在要做什麼　▶　照 Step 2 的四個步驟，自己把第二個錯處理完

提示（先自己想，想不到再往下看）

提示 ①　它藏在「某一個特定的等級」。
你在 Step 1 已經看到線索了。

提示 ②　把 7 個等級都選一次，
只看「每月給付上限」這一個數字。

提示 ③　兩個相鄰的等級，顯示了同一個上限。

你應該看到什麼

你會發現：選第 8 級時，
上限顯示 32,090——
跟第 7 級一模一樣。
但基準上第 8 級應該是 36,180 。

找到之後，照 Step 2-2 → 2-3 → 2-4
的順序自己走一遍。

沒看到的話，先檢查這幾件事

① 七個等級都試了，看不出問題
→ 把七個上限抄下來跟基準對照
10,020 ／ 15,460 ／ 18,580 ／ 24,100 ／
28,070 ／ 32,090 ／ 36,180
② 找到了但不知道怎麼描述
→ 照 Step 2-3 的四要素格式改寫

為什麼這個 bug 只有試邊界才找得到：中間的等級全部都是對的。如果你只試第 3 、 4 、 5 級（多數人的習慣），這個錯會一路跟著上線——而它影響的正好是失能最嚴重的那一群個案。

Bug 2 的答案與說明（自己找到了再看）

項目	內容
位置	calculator.js　lookupPayment() 函式
原本的寫法	const capped = Math.min(level, 7);　← 上限被限制在 7 return AMOUNTS[capped];
為什麼會這樣	AI 想加一個防呆（避免超出陣列範圍），但把上限寫成 7 而不是 8。 這叫 off-by-one，是最常見的一類錯誤——語法完全正確，只差一個數字
建議的修法	不要只把 7 改成 8。更好的做法是改成明確的範圍檢查： if (level < 2 || level > 8) throw new Error('CMS 等級必須介於 2 到 8 之間 '); 這樣超出範圍時會明確報錯，而不是安靜地回傳一個錯的數字
怎麼確認修好了	七個等級各點一次，對照基準的七個數字。 並確認低收那一格仍然是 0 （Bug 1 沒有被改壞）

> 「建議的修法」那一列值得多想一下：把 7 改成 8 可以讓這個 bug 消失，但下次有人輸入第 9 級時，它還是會安靜地回傳一個錯的數字。改成明確報錯，是把「安靜的錯」變成「吵鬧的錯」——後者好處理得多。

自己處理一個 bug 的標準流程（Bug 2、Bug 3 都照這個走）

步驟	要做什麼	做完要能回答
① **發現**	點過每一種情況，找出跟你預期不一樣的那一個	「我做了什麼操作，看到什麼結果？」
② **確認我是對的**	查出正確答案與依據（給付基準、規格書）	「正確答案是多少？我的依據是什麼、出處在哪？」
③ **縮小範圍**	找出「會錯」與「不會錯」的分界點；有紅字就去 Console 撈	「哪一種輸入會錯、哪一種不會？」
④ **描述給 AI**	現象 ＋ 證據 ＋ 正確答案 ＋ 範圍限制，四要素齊全	「我貼給它的那段話，四個要素都有嗎？」
⑤ **確認修好**	要它說明改了哪幾行；自己點一次； 再點一個不該被影響的組合	「不該被改壞的地方，我實際確認過了嗎？」
⑥ **記錄**	填進除錯紀錄四欄表	「三個月後我看得懂當初為什麼這樣改嗎？」

這六步就是 Step 2 帶你走過的那一遍。Bug 2 與 Bug 3 請照著自己走——特別是第 ② 步與第 ⑤ 步，它們是最容易被跳過、也最容易出事的兩步。

Step 4　自己找：第三個 bug（要開 F12 才找得到）

你現在要做什麼　▶　回到 Step 1-2 你看到的那段紅字，現在來處理它

重現步驟

1. 重新整理頁面（讓畫面回到初始狀態）
2. F12 → Console → 清空
3. 身分別留在「請選擇」，直接按計算
4. 看畫面：完全沒反應
5. 看 Console：出現紅字

更陰險的版本，也試一次

1. 先正常算一次（第 5 級 + 一般戶）
2. 把身分別改回「請選擇」
3. 再按一次計算

畫面會維持上一次的舊數字不變——
使用者完全不會發現自己看的是過期結果

你應該看到什麼

Console 出現：
Uncaught TypeError: Cannot read properties of undefined (reading 'rate')
at calculate (calculator.js:42:26)
照第 11 頁的順序讀：
② reading 'rate' → ③ 第 42 行 → ① TypeError

沒看到的話，先檢查這幾件事

① Console 沒有紅字
→ 先按垃圾桶清空，再操作一次
② 紅字太多看不出哪一則
→ 清空後只做「按計算」這一個動作
③ 看不懂那段英文
→ 整段貼給 AI，問「這是什麼意思？」

「更陰險的版本」請務必試一次。它示範了一件重要的事：同一個 bug 可以有兩種症狀，一種明顯（沒反應）、一種幾乎不可能察覺（顯示舊數字）。而後者才是真實世界裡比較常見的樣子。

Step 4-2　把整段錯誤訊息貼給 AI

你現在要做什麼　▶　照下面的格式，把證據完整貼上

可以直接照抄（把〔〕換成你實際看到的）

我在測試長照給付試算器時遇到一個問題。

重現步驟：身分別留在「請選擇」不選，直接按「計算」。
現象：畫面完全沒有反應，沒有任何提示訊息。
　　如果先算過一次，畫面會維持上一次的舊數字不變。

Console 的錯誤訊息：
〔把整段紅字貼在這裡，含 at ... 那幾行〕

我希望：沒選身分別時，畫面上要出現看得懂的提示，
　　而不是靜默無反應。
請只修改 calculator.js 與 index.html。

你應該看到什麼

它會指出： IDENTITY_RATES[''] 查不到，
回傳 undefined，接著讀 .rate 就爆掉。
修法是在讀 .rate 之前先檢查，
　並在畫面上顯示提示訊息。
修好之後：不選身分別按計算，
畫面出現「請先選擇身分別」。

沒看到的話，先檢查這幾件事

① 它只改了 calculator.js，畫面還是沒提示
　→ 要它補上：「錯誤訊息要顯示在畫面上，
　　不能只丟到 Console」
② 它把下拉選單的「請選擇」拿掉了
　→ 這是規避不是修正。要它保留，
　　改成沒選時給提示

第 ② 個陷阱很值得注意： AI 有時會用「讓問題不可能發生」的方式來「修好」它（例如直接預設選第一項）。這確實讓症狀消失了，但也悄悄改變了你的設計。每次修完都要確認：它是修好了，還是繞過去了？

Bug 3 修好前後：使用者實際看到的差別

修好前

修好前
身分別：請選擇　→ 按計算
畫面：完全沒有變化
或：維持上一次的舊數字
Console：一段英文紅字
使用者：不知道發生什麼事

修好後

修好後
身分別：請選擇　→ 按計算
畫面：出現紅色提示
「請先選擇身分別」
Console：沒有紅字
使用者：知道下一步要做什麼

這個 bug 的修法重點不是「不要出錯」

而是「出錯時要說人話」。程式在什麼情況下都可能遇到不完整的輸入——重點是讓使用者知道發生了什麼、下一步該做什麼

為什麼沉默失敗特別糟

使用者會以為系統壞了，或更糟——以為看到的數字是有效的。醫療場域裡，後者可能直接影響判斷

怎麼檢查你的專案有沒有這種問題

把每一個輸入欄位都留空一次、都填奇怪的值一次，看畫面有沒有給出看得懂的回應

這一項會出現在第 3 週的檢核表

「錯誤訊息使用者看得懂嗎」就是這一項。現在修好，下週檢查時直接過關

三個 bug 到這裡全部處理完。請把 21 種組合再點一次確認——特別是第 5 級低收（應為 0）與第 8 級一般戶（上限 36,180、自負額 5,789）。

Step 5　附帶任務：讓畫面在手機上不破版

你現在要做什麼　▶　切到手機模式（寬度 390），把破版的地方修好

先確認問題（用 Elements 分頁）

1. 開發者工具 → 點左上角箭頭圖示
2. 點畫面上那張白色卡片
3. 右邊會顯示它套用的 CSS

你會看到　`.card { width: 720px; }`

可以直接照抄的指令

這個頁面在手機上（寬度 390px）會超出螢幕。
請修正，讓它在手機與桌機上都能正常顯示。

我知道 `.card` 用了固定寬度 720px，
而且 `<head>` 裡沒有 `viewport` 設定。
請只修改 `index.html`。

你應該看到什麼

卡片會自動縮到螢幕寬度內，
兩個下拉選單在窄螢幕時
各佔一行而不是擠在一起。
文字大小也變得看得清楚。

記得也回桌機寬度看一次，
確認沒有被改壞。

沒看到的話，先檢查這幾件事

① 手機版好了，桌機版卻變很窄
→ 要它用 max-width 而不是 width
② 還是要左右拖曳
→ 檢查 `<head>` 有沒有 viewport 那一行
③ 選單還是擠在一起
→ 要它給 .row 加 flex-wrap

這一題示範了「你不必懂 CSS，但要看得懂線索」：Elements 分頁告訴你 width 是 720px，你就能在指令裡寫出「我知道 .card 用了固定寬度」——AI 收到這一句，修起來又快又準。這就是證據的價值。

手機版要檢查的五件事（每一次交付前都做一次）

#	檢查什麼	怎麼看	常見問題
1	有沒有超出螢幕	手機模式下看有沒有左右捲軸	固定寬度（ width: 720px ）沒改成 max-width
2	有沒有 viewport 設定	看 <head> 裡有沒有 viewport 那一行	沒有的話手機會用桌機寬度縮小整頁，字會小到看不清
3	字看得清楚嗎	手機模式下實際讀一次	字級寫死成很小的 px ； 或縮放後全部變小
4	按鈕點得到嗎	看按鈕夠不夠大、有沒有被切掉	高齡使用者需要更大的點擊區域
5	用真的手機開一次	把檔案傳到手機，或等第 4 週部署後用網址開	模擬器不等於真機。字型、捲動、鍵盤都可能不一樣

> 第 5 項在這一週還做不到（東西還沒部署），但請先記著。第 4 週把網址做出來之後，用自己的手機開一次是驗收條件之一——那一次通常還會再抓到一兩個問題。

Step 6　三輪真實的迭代：完整的對話節奏

輪次	你說了什麼	它做了什麼	你怎麼確認
第一輪 **先確認** **基本功能**	「我選第 5 級 + 低收，自負額顯示 3,856。 低收的部分負擔比率是 0%，應該顯示 0。 請只改 calculator.js，動手前先說明問題在哪。」	指出 low 的 rate 寫成 0.05， 應為 0。改了一行	① 要它說改了哪幾行 ② 第 5 級低收 → 顯示 0 ③ 第 5 級一般戶 → 仍是 3,856
第二輪 **再處理** **邊界**	「選第 8 級時，每月給付上限顯示 32,090， 跟第 7 級一樣。基準上第 8 級是 36,180。」	指出 Math.min(level, 7) 上限寫錯。建議改成 明確的範圍檢查	① 七個等級全部對照基準 ② 低收那格仍是 0 ③ 輸入第 9 級會明確報錯
第三輪 **最後** **處理沉默**	「不選身分別按計算，畫面完全沒反應。 Console 顯示：〔整段錯誤訊息〕 希望畫面上出現看得懂的提示。」	加上檢查與畫面提示， 改了 calculator.js 與 index.html	① 不選身分別 → 出現提示 ② Console 沒有紅字 ③ 21 種組合重點一次

注意三輪的共同結構：一輪只處理一個問題，每一輪結束都做三項確認，而且第三項永遠是「不該被影響的地方有沒有被改壞」。三個問題一起丟給 AI 會快一點，但你會完全失去這個確認的能力。

為什麼一次只修一個問題

① 你才分得清是哪一句話造成哪個改變

- 三個問題一起丟，它可能改了七、八個地方
- 其中哪些是必要的、哪些是它順手改的，你完全無從判斷
- 出問題時也無法回推是哪一個修改造成的

② 「改 A 壞 B」會當場現形

- 一次一個，每次都確認「不該被影響的地方」還正常
- 一次三個的話，等你發現一般戶也變成 0 時，已經很難查是哪一步造成的
- 這也是第 3 週要把這個確認自動化成測試的原因

③ 你的除錯紀錄才寫得出來

- 四欄表的每一列對應一個問題。混在一起就填不了
- 而這份紀錄是本週的主要交付物
- 更重要的是：它逼你把「我做了什麼」講清楚，這是往後排查問題的基本功

唯一的例外：如果三個問題明顯是同一個原因造成的（例如所有金額都少了一個位數），那就一起講。判斷標準是「它們會不會是同一行程式碼的問題」。

回頭看：三個 bug 分別是靠什麼找到的

	Bug 1　低收自負額	Bug 2　第 8 級上限	Bug 3　沒選身分別
故障樣態	數字錯（最危險）	數字錯（最危險）	會報錯 + 沉默失敗
Console 有紅字嗎	沒有	沒有	有
靠什麼找到的	知道低收部分負擔是 0%	把七個等級都試一次	按 F12 看 Console
需要看程式碼嗎	不需要	不需要	不需要
學到什麼	領域知識比程式能力重要	邊界值最容易錯	沒線索時，證據在 Console 裡

三個 bug 你全部都修好了，而且從頭到尾沒有真的「讀懂」那份程式碼——這是刻意的安排。這一週要建立的是「不懂程式也能除錯」的信心；下一週才學怎麼讀，以及怎麼把這些手動確認變成自動化的測試。

Step 7　填除錯紀錄四欄表

你現在要做什麼　▶　三個 bug 各填一列，這是本週的主要交付物

四欄各寫什麼

① 現象　　我做了什麼操作，看到什麼結果
② 證據　　實際貼上的錯誤訊息或數字，
　　不可以寫「壞掉了」「怪怪的」
③ 我說了什麼　貼上你給 AI 的那段話
④ 結果　　它改了哪幾行，我怎麼確認修好了

已填好的一列（照這個深度寫）

① 選第 5 級 + 低收入戶，自負額顯示 NT$ 1,205
② 給付基準：低收入戶部分負擔 0%， 24,100 × 0% = 0
③ 「現象：…應該是：…請只改 calculator.js」（全文）
④ 改 IDENTITY_RATES 低收 rate 0.05→0，一行。
　確認：低收顯示 0；一般戶仍為 3,856 未受影響

你應該看到什麼

三列都填滿，四欄都有內容。
第 ② 欄必須是實際的訊息或數字。
第 ④ 欄必須包含
「不該被影響的地方沒有被改壞」
這一項確認。
兩者少一個就視為未完成。

沒看到的話，先檢查這幾件事

① 忘記當時貼給 AI 的原文了
　→ 回對話紀錄複製。下次邊做邊記
② 第 ④ 欄不知道寫什麼
　→ 就寫你實際點了哪幾種組合、
　　分別看到什麼數字

這份表的價值不只在交作業。三個月後你回頭看自己的專案，這份紀錄會告訴你「當初為什麼要這樣改」——那是任何程式碼註解都寫不出來的資訊。

除錯紀錄四欄表：空白樣板（可直接抄到你的文件裡）

① 現象	② 證據	③ 我跟 AI 說了什麼	④ 結果與確認
我做了什麼操作， 看到什麼結果	實際貼上的錯誤訊息、 或實際的數字與正確答案的出處	貼上你給 AI 的那段話 （可節錄，但四要素要看得出來）	它改了哪幾行； 我點了哪些組合確認，看到什麼

第一列是欄位說明，下面三列填你的三個 bug 。② 欄寫「壞掉了」「怪怪的」視為未完成；④ 欄沒有「不該被影響的地方也確認過」這一項，同樣視為未完成。

除錯紀錄四欄表：填好的完整範例（照這個深度寫）

① 現象	② 證據	③ 我說了什麼	④ 結果與確認
選第 5 級＋低收入戶， 按計算後自負額顯示 NT$ 1,205	長照給付基準：低收入戶 部分負擔 0%。 24,100 × 0% ＝ 0 （出處：範例 README）	「現象：第 5 級＋低收， 自負額顯示 1,205。 應該是：低收比率 0%，應為 0。 請只改 calculator.js， 動手前先說明問題在哪。」	改 IDENTITY_RATES 低收 rate 0.05 → 0，共一行。 確認：低收顯示 0； 一般戶仍為 3,856 未受影響
選第 8 級，每月給付上限 顯示 32,090， 跟第 7 級一樣	基準：第 8 級為 36,180。 七個等級逐一比對後 只有第 8 級對不上	「選第 8 級時上限顯示 32,090， 跟第 7 級一樣。基準是 36,180。 請只改 calculator.js。」	lookupPayment 的 Math.min(level, 7) 改成明確範圍檢查（2–8）。 確認：七個等級全部對； 低收那格仍是 0
身分別留「請選擇」按計算， 畫面完全沒反應。 先算過一次的話， 會維持舊數字不變	Console： Uncaught TypeError: Cannot read properties of undefined (reading 'rate') at calculate (calculator.js:42:26)	「重現步驟：不選身分別按計算。 現象：畫面沒反應。 Console：〔整段錯誤訊息〕 希望畫面出現看得懂的提示。」	calculator.js 加檢查並丟出「請先選擇身分別」； index.html 加訊息區塊。 確認：出現提示、Console 無紅字、21 種組合重點一次都對

注意每一列的第 ④ 欄，最後一句都是「不該被影響的地方也確認過」。這一句是本週的核心習慣——下一週它會變成 test.html 裡自動跑的 29 項，三秒就驗完。

卡住了？照症狀查（一）畫面與數字

症狀	先做這件事	可能原因	怎麼修
按了完全沒反應	F12 → Console 看有沒有紅字	有紅字：程式中途出錯 沒紅字：按鈕沒接上	有紅字 → 整段貼給 AI 沒紅字 → 問「按鈕的事件有接上嗎」
數字顯示的是上一次的結果	重新整理再操作一次	這一次的計算中途失敗了，畫面沒更新	看 Console。要求「失敗時要清掉舊結果並顯示提示」
數字不對，但沒有任何錯誤	自己手算一次	計算邏輯錯（最常見） 或顯示格式錯	用「現象 + 證據 + 正確答案」三要素描述給 AI
改完某一處，別的地方壞了	確認它到底改了哪些檔案	AI 順手改了範圍外的東西	要它列出所有改動並還原多餘的。 下次在指令裡寫明範圍限制
畫面一片空白	F12 → Console	多半是 SyntaxError，整份檔案都沒執行	貼上錯誤訊息與那一行前後五行。 通常是少一個括號或引號
手機上破版	Ctrl + Shift + M 切到寬度 390	固定寬度、 或缺 viewport 設定	用 Elements 分頁找出是哪一條 CSS， 把線索寫進指令裡

使用方式：卡住時先找到你的「症狀」，照「先做這件事」實際做一次，再往右看。不要跳過中間那一步——沒有先確認原因就直接修，通常會修到錯的地方。

卡住了？照症狀查（二）跟 AI 的互動

症狀	可能原因	怎麼處理
它說「已修正」，但問題還在	它改的不是造成問題的地方	要它說明「你改了哪幾行、為什麼認為那是原因」。 它答不出所以然，通常就是猜的
來回三次都沒改對	你給的證據不夠，它一直在猜	停下來，補齊三件事：完整錯誤訊息、重現步驟、 正確答案應該是多少。通常補完就會中
它一直改壞別的地方	沒有限制範圍	每一次指令都加：「請只修改〔檔名〕， 不要更動其他檔案。完成後說明你改了什麼。」
它把問題「繞過去」了	例如直接拿掉「請選擇」選項， 讓錯誤不可能發生	這是規避不是修正。指出來，並說明 「請保留原本的設計，改成出錯時給提示」
它前後說法不一致	對話太長，忘了前面的內容	開新對話，重貼完整的檔案再問。 這是 M7 講的上下文長度限制
越改越糟，回不去了	沒有可以回去的版本	本週先用「另存一份」的土法。 第 4 週會教 Git 的 revert，那是正規做法

最後一列請特別注意：這一週還沒學版本控制，所以每次要 AI 做較大的修改之前，建議先把整個資料夾複製一份備份。土法，但有效——下週就有正規做法了。

常見的四個誤解（做完之後回頭看一次）

誤解	為什麼是錯的	正確的做法
① 三個問題 一起講比較快	它可能改了七八個地方， 你分不清哪個改動對應哪個問題	一次一個，每次做三項確認。 看起來慢，實際上比來回釐清快得多
② 沒有紅字 就是沒問題	三個 bug 裡只有一個 會留下紅字	另外兩個安靜地顯示錯的金額。 沒有錯誤訊息的錯，只有你抓得到
③ 它說已修正 就是修好了	它沒有執行過那段程式碼， 也沒有看過你的畫面	要它說明改了哪幾行；自己回畫面點一次； 再點一個不該被影響的組合
④ 除錯要 看得懂程式碼	完全不必。三個 bug 你都是從畫面與訊息找到的	你需要的是：知道正確答案、會看 Console 、 會描述現象。三件都跟語法無關

第 ④ 個誤解值得多說一句：這一週你修好了三個 bug ，而且從頭到尾沒有真的「讀懂」程式碼。這是刻意的安排——先建立「不懂程式也能除錯」的信心，下一週再學怎麼讀。

本週自我檢核（答不出來請回去看標註的頁數）

Q 1

一則錯誤訊息有哪四個部位？建議先看哪一個？

答不出來 → 回到第 11 頁的錯誤訊息解剖

Q 2

「給證據不要給形容詞」——一句好的問題描述要有哪三個要素？

現象、證據、正確答案應該是多少 → 第 9 頁與第 30 頁

Q 3

三個 bug 裡，哪一個 Console 完全不會有紅字？為什麼那種最危險？

答不出來 → 回到第 18 頁的三種故障樣態

Q 4

它說「已修正」之後，你要做哪三件事才算確認？

要它說明改了哪幾行／自己點一次／點一個不該被影響的組合 → 第 31 頁

第四題是本週最重要的一題。「點一個不該被影響的組合」這個習慣，就是第 3 週要自動化成 test.html 的東西——現在用手做過，下週你才會明白測試在替你省什麼。

本週交付與驗收條件

交付物	驗收條件（全部達成才算通過）	怎麼查
① 除錯紀錄四欄表	三個 bug 各一列，四欄都有內容。 證據欄必須是實際的錯誤訊息或數字，不可寫「壞掉了」。 結果欄必須包含「不該被影響的地方沒被改壞」的確認	文件檢查
② 修好的試算器	三個 bug 全部修好， 21 種組合都算得對	當場操作，含第 2 級與第 8 級；對照 範例 \v0.2
③ 手機版	寬度 390px 下畫面不破版、不需左右拖曳	開發者工具切手機模式看一次
④ 口頭一句	能在 Console 指出一則錯誤訊息的檔名與行號	自我檢核第 1 題

交出去之前，請把 21 種組合再點一次。特別確認第 5 級低收（自負額 0）與第 8 級一般戶（上限 36,180、自負額 5,789）——這兩格是本週修正的重點。

第 2 週重點回顧

1. AI 看不到你的螢幕，你是它唯一的感官
2. 給證據不給形容詞：現象 + 證據 + 正確答案應該是多少
3. 一則錯誤訊息只有四個部位。先看②具體訊息，不要從第一行讀起
4. 沒有錯誤訊息的錯最危險——只有「知道正確答案」的人抓得到
5. 一次只修一個問題，每次都確認「不該被影響的地方」還正常
6. 除錯不需要看得懂程式碼。你今天就是這樣修好三個 bug 的

下一步　下一週：驗證——你怎麼知道 AI 寫的是對的？我們會把「點一個不該被影響的組合」這件事，變成三秒跑完 29 項的測試。